1. Aim

To develop a simple Hospital Management System using HTML and CSS for the frontend and Core Java for the backend, allowing hospital staff to log in, manage patients and doctors, book appointments and view a dashboard. Records are stored in Java ArrayList and HashMap.

2. Technologies

- Frontend: HTML, CSS
- Backend: Core Java (built-in HTTP server)
- Data storage: Java ArrayList and HashMap
- Communication: HTML form requests to Java HTTP server

3. Features

- Staff login with username and password
- Dashboard with patient, doctor and appointment counts
- Today's appointments shown on the dashboard
- Add, view and delete patients
- Add, view and delete doctors
- Book, view and delete appointments (patient and doctor picked from a list)
- Every field checked before saving, pages locked until login

4. Project Structure

```
HospitalManagement/
|-- Hospital.java
`-- style.css
```

5. Demo Login Credentials

Username	Password	Access
admin	admin123	Full access
reception	reception123	Full access

6. style.css — Frontend Styling

```
* { box-sizing: border-box; }
body { margin: 0; font: 15px/1.5 system-ui, sans-serif;
       background: #f3f6f9; color: #1f2933; }
body.center { min-height: 100vh; display: grid; place-items: center; padding: 16px; }

header { display: flex; flex-wrap: wrap; gap: 12px; justify-content: space-between;
         align-items: center; padding: 12px 24px; background: #0f766e; color: #fff; }
nav a { color: #fff; text-decoration: none; margin-left: 18px; opacity: .9; }
nav a:hover { opacity: 1; text-decoration: underline; }

main { max-width: 1000px; margin: 0 auto; padding: 24px 16px; }
h1 { margin: 0 0 16px; font-size: 24px; }
h2 { font-size: 18px; margin: 28px 0 10px; }

.card { background: #fff; border-radius: 10px; padding: 20px;
        box-shadow: 0 1px 3px rgba(0,0,0,.08); margin-bottom: 20px; }
.login { width: 100%; max-width: 360px; display: grid; gap: 12px; }
.grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(170px, 1fr));
        gap: 12px; margin-bottom: 14px; }

label { display: grid; gap: 4px; font-size: 13px; font-weight: 600; color: #52606d; }
input, select { font: inherit; padding: 8px 10px; border: 1px solid #cbd2d9;
                border-radius: 6px; background: #fff; }
input:focus, select:focus { outline: 2px solid #14b8a6; border-color: transparent; }

button { font: inherit; padding: 8px 16px; border: 0; border-radius: 6px;
         background: #0f766e; color: #fff; cursor: pointer; }
button:hover { background: #115e59; }
button.danger { background: #fff; color: #b91c1c; border: 1px solid #fca5a5;
                padding: 4px 10px; }
button.danger:hover { background: #fee2e2; }

table { width: 100%; border-collapse: collapse; background: #fff;
        border-radius: 10px; overflow: hidden; box-shadow: 0 1px 3px rgba(0,0,0,.08); }
th, td { padding: 10px 12px; text-align: left; border-bottom: 1px solid #e4e7eb; }
th { background: #f8fafc; font-size: 13px; color: #52606d; }
td form { margin: 0; }

.stats { display: grid; grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
         gap: 16px; }
.stat { background: #fff; border-radius: 10px; padding: 18px; text-decoration: none;
        color: #52606d; box-shadow: 0 1px 3px rgba(0,0,0,.08);
        border-left: 4px solid #14b8a6; }
.stat b { display: block; font-size: 30px; color: #0f766e; }

.error { color: #b91c1c; background: #fee2e2; padding: 8px 12px;
         border-radius: 6px; margin: 0 0 12px; }
.muted { color: #7b8794; font-size: 13px; }
code { background: #f1f5f9; padding: 1px 5px; border-radius: 4px; }
@media (max-width: 640px) {
```

```
  nav a { margin: 0 12px 0 0; }
  table { display: block; overflow-x: auto; }
}
```

7. Login Page HTML — Frontend

```
<!doctype html>
<html>
<head>
  <meta charset=utf-8>
  <meta name=viewport content='width=device-width,initial-scale=1'>
  <title>Login · CityCare Hospital</title>
  <link rel=stylesheet href=/style.css>
</head>
<body class=center>
<form method=post action=/login class='card login'>
  <h1>CityCare Hospital</h1>
  <label>Username<input name=username required autofocus></label>
  <label>Password<input name=password type=password required></label>
  <button>Log in</button>
  <p class=muted>Demo: <code>admin / admin123</code> or
    <code>reception / reception123</code></p>
</form>
</body>
</html>
```

8. Hospital.java — Java Backend

```
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpServer;

import java.io.IOException;
import java.io.OutputStream;
import java.math.BigInteger;
import java.net.InetSocketAddress;
import java.net.URLDecoder;
import java.nio.file.Files;
import java.nio.file.Path;
import java.security.SecureRandom;
import java.time.LocalDate;
import java.util.*;

import static java.nio.charset.StandardCharsets.UTF_8;

// Run: java Hospital.java   then open http://localhost:8080
// ponytail: in-memory store, data resets on restart; swap for a file/DB
// when it must persist.
// Default HttpServer executor is single-threaded, so plain collections are safe.
public class Hospital {
    static final Map<String, String> USERS =
            Map.of("admin", "admin123", "reception", "reception123");
    static final Map<String, String> SESSIONS = new HashMap<>();
    static final SecureRandom RNG = new SecureRandom();

    // entity -> "Label|type". type: html input type, "a,b,c" for fixed options,
    // "@entity" to pick from another entity
    static final Map<String, String[]> FIELDS = new LinkedHashMap<>();
    static final Map<String, List<Map<String, String>>> DATA = new HashMap<>();
    static int nextId = 1;

    public static void main(String[] args) throws IOException {
        FIELDS.put("patients", new String[]{"Name|text", "Age|number",
                "Gender|Male,Female,Other", "Phone|tel", "Condition|text"});
        FIELDS.put("doctors", new String[]{"Name|text", "Specialty|text", "Phone|tel"});
        FIELDS.put("appointments", new String[]{"Patient|@patients",
                "Doctor|@doctors", "Date|date", "Time|time", "Reason|text"});
        FIELDS.keySet().forEach(e -> DATA.put(e, new ArrayList<>()));

        String today = LocalDate.now().toString();
        add("doctors", "Dr. Anita Rao", "Cardiology", "9876500001");
        add("doctors", "Dr. Vikram Shah", "Orthopedics", "9876500002");
        add("doctors", "Dr. Meera Iyer", "Pediatrics", "9876500003");
        add("patients", "Rahul Kumar", "45", "Male", "9123400001", "Hypertension");
        add("patients", "Priya Nair", "32", "Female", "9123400002", "Knee pain");
        add("patients", "Aarav Singh", "7", "Male", "9123400003", "Fever");
        add("appointments", "Rahul Kumar", "Dr. Anita Rao", today, "10:00",
                "BP check-up");
        add("appointments", "Aarav Singh", "Dr. Meera Iyer", today, "11:30",
```

```
            "Follow-up");

    HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
    server.createContext("/", Hospital::handle);
    server.start();
    System.out.println("Hospital app running at http://localhost:8080"
            + "  (login: admin / admin123)");
}

static void add(String entity, String... values) {
    Map<String, String> row = new LinkedHashMap<>();
    row.put("id", String.valueOf(nextId++));
    String[] fields = FIELDS.get(entity);
    for (int i = 0; i < fields.length; i++) row.put(label(fields[i]), values[i]);
    DATA.get(entity).add(row);
}

static void handle(HttpExchange ex) throws IOException {
    String path = ex.getRequestURI().getPath();
    boolean post = ex.getRequestMethod().equals("POST");
    String body = post ? new String(ex.getRequestBody().readAllBytes(), UTF_8) : null;
    Map<String, String> form = parse(body);

    if (path.equals("/style.css")) {
        send(ex, 200, "text/css", Files.readString(Path.of("style.css")));
        return;
    }
    if (path.equals("/login")) {
        String user = form.get("username"), pass = form.get("password");
        if (post && pass != null && pass.equals(USERS.get(user))) {
            String token = new BigInteger(130, RNG).toString(32);
            SESSIONS.put(token, user);
            // SameSite=Strict also blocks cross-site form posts (CSRF)
            ex.getResponseHeaders().add("Set-Cookie",
                    "sid=" + token + "; HttpOnly; Path=/; SameSite=Strict");
            redirect(ex, "/");
        } else {
            String error = post ? "Invalid username or password" : null;
            send(ex, 200, "text/html", loginPage(error));
        }
        return;
    }

    String user = SESSIONS.get(sessionId(ex));
    if (user == null) {
        redirect(ex, "/login");
        return;
    }
    if (path.equals("/logout")) {
        SESSIONS.remove(sessionId(ex));
        redirect(ex, "/login");
```

```
            return;
        }
        if (path.equals("/")) {
            send(ex, 200, "text/html", page("Dashboard", user, dashboard()));
            return;
        }

        String[] parts = path.split("/"); // "/patients" or "/patients/delete"
        String entity = parts.length > 1 ? parts[1] : "";
        if (!FIELDS.containsKey(entity)) {
            send(ex, 404, "text/html", page("Not found", user, "<p>Page not found.</p>"));
            return;
        }
        if (post && parts.length == 3 && parts[2].equals("delete")) {
            DATA.get(entity).removeIf(r -> r.get("id").equals(form.get("id")));
            redirect(ex, "/" + entity);
            return;
        }
        String error = null;
        if (post) {
            String[] fields = FIELDS.get(entity);
            String[] values = new String[fields.length];
            for (int i = 0; i < fields.length; i++) {
                values[i] = form.getOrDefault(label(fields[i]), "");
                if (values[i].isBlank()) error = label(fields[i]) + " is required";
            }
            if (error == null) {
                add(entity, values);
                redirect(ex, "/" + entity);
                return;
            }
        }
        String html = addForm(entity, error) + table(entity, DATA.get(entity), true);
        send(ex, error == null ? 200 : 400, "text/html", page(title(entity), user, html));
    }

    static String dashboard() {
        StringBuilder sb = new StringBuilder("<div class=stats>");
        for (String e : FIELDS.keySet())
            sb.append("<a class=stat href=/").append(e).append("><b>")
                    .append(DATA.get(e).size()).append("</b>").append(title(e))
                    .append("</a>");
        String today = LocalDate.now().toString();
        List<Map<String, String>> todays = DATA.get("appointments").stream()
                .filter(r -> today.equals(r.get("Date"))).toList();
        return sb.append("</div><h2>Today's appointments</h2>")
                .append(table("appointments", todays, false)).toString();
    }

    static String addForm(String entity, String error) {
        StringBuilder sb = new StringBuilder("<form method=post class=card>");
```

```
        if (error != null) sb.append("<p class=error>").append(esc(error)).append("</p>");
        sb.append("<div class=grid>");
        for (String spec : FIELDS.get(entity)) {
            String name = label(spec), type = spec.split("\\|")[1];
            sb.append("<label>").append(name);
            if (type.startsWith("@") || type.contains(",")) {
                List<String> options = type.startsWith("@")
                        ? DATA.get(type.substring(1)).stream()
                                .map(r -> r.get("Name")).toList()
                        : List.of(type.split(","));
                sb.append("<select name=").append(name)
                        .append(" required><option value=''>Select…</option>");
                for (String o : options)
                    sb.append("<option>").append(esc(o)).append("</option>");
                sb.append("</select>");
            } else {
                sb.append("<input name=").append(name).append(" type=").append(type)
                        .append(type.equals("number") ? " min=0 max=150" : "")
                        .append(" required>");
            }
            sb.append("</label>");
        }
        return sb.append("</div><button>Add ")
                .append(title(entity).replaceAll("s$", ""))
                .append("</button></form>").toString();
    }

    static String table(String entity, List<Map<String, String>> rows,
                        boolean deletable) {
        if (rows.isEmpty()) return "<p class=muted>No records.</p>";
        StringBuilder sb = new StringBuilder("<table><tr><th>#</th>");
        for (String spec : FIELDS.get(entity))
            sb.append("<th>").append(label(spec)).append("</th>");
        if (deletable) sb.append("<th></th>");
        sb.append("</tr>");
        for (Map<String, String> r : rows) {
            sb.append("<tr>");
            r.values().forEach(v -> sb.append("<td>").append(esc(v)).append("</td>"));
            if (deletable)
                sb.append("<td><form method=post action=/").append(entity)
                        .append("/delete><input type=hidden name=id value=")
                        .append(r.get("id"))
                        .append("><button class=danger>Delete</button></form></td>");
            sb.append("</tr>");
        }
        return sb.append("</table>").toString();
    }

    static String page(String title, String user, String body) {
        return "<!doctype html><html><head><meta charset=utf-8>"
                + "<meta name=viewport content='width=device-width,initial-scale=1'>"
```

```
            + "<title>" + title + " · CityCare Hospital</title>"
            + "<link rel=stylesheet href=/style.css></head><body>"
            + "<header><b>CityCare Hospital</b><nav>"
            + "<a href=/>Dashboard</a><a href=/patients>Patients</a>"
            + "<a href=/doctors>Doctors</a><a href=/appointments>Appointments</a>"
            + "<a href=/logout>Logout (" + esc(user) + ")</a></nav></header>"
            + "<main><h1>" + title + "</h1>" + body + "</main></body></html>";
    }

    static String loginPage(String error) {
        return "<!doctype html><html><head><meta charset=utf-8>"
            + "<meta name=viewport content='width=device-width,initial-scale=1'>"
            + "<title>Login · CityCare Hospital</title>"
            + "<link rel=stylesheet href=/style.css></head><body class=center>"
            + "<form method=post action=/login class='card login'>"
            + "<h1>CityCare Hospital</h1>"
            + (error == null ? "" : "<p class=error>" + error + "</p>")
            + "<label>Username<input name=username required autofocus></label>"
            + "<label>Password<input name=password type=password required>"
            + "</label><button>Log in</button>"
            + "<p class=muted>Demo: <code>admin / admin123</code> or "
            + "<code>reception / reception123</code></p></form></body></html>";
    }

    static String label(String spec) { return spec.split("\\|")[0]; }

    static String title(String entity) {
        return Character.toUpperCase(entity.charAt(0)) + entity.substring(1);
    }

    static String esc(String s) {
        return s.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")
                .replace("\"", "&quot;").replace("'", "&#39;");
    }

    static Map<String, String> parse(String body) {
        Map<String, String> m = new HashMap<>();
        if (body != null)
            for (String kv : body.split("&")) {
                int i = kv.indexOf('=');
                if (i > 0) m.put(URLDecoder.decode(kv.substring(0, i), UTF_8),
                        URLDecoder.decode(kv.substring(i + 1), UTF_8).trim());
            }
        return m;
    }

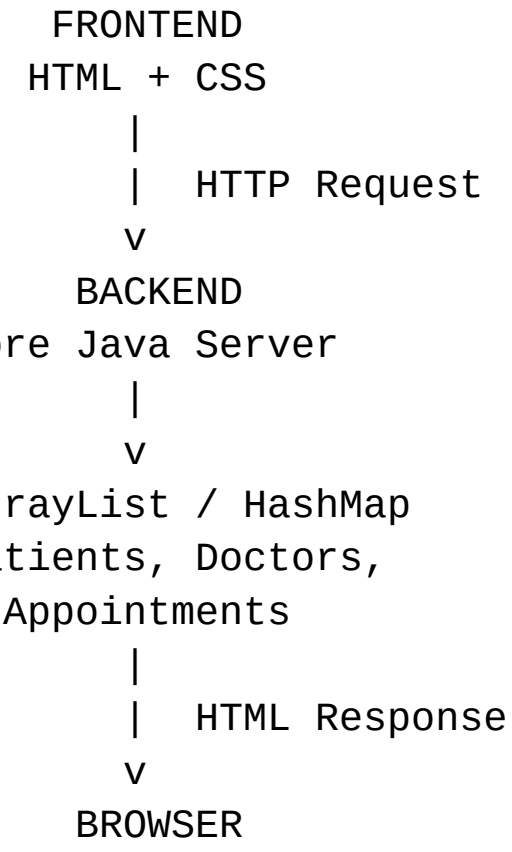
    static String sessionId(HttpExchange ex) {
        String cookie = ex.getRequestHeaders().getFirst("Cookie");
        if (cookie != null)
            for (String c : cookie.split(";\\s*"))
                if (c.startsWith("sid=")) return c.substring(4);
```

```
        return "";
    }

    static void send(HttpExchange ex, int code, String type, String body)
            throws IOException {
        byte[] bytes = body.getBytes(UTF_8);
        ex.getResponseHeaders().set("Content-Type", type + "; charset=utf-8");
        ex.sendResponseHeaders(code, bytes.length);
        try (OutputStream out = ex.getResponseBody()) { out.write(bytes); }
    }

    static void redirect(HttpExchange ex, String to) throws IOException {
        ex.getResponseHeaders().set("Location", to);
        ex.sendResponseHeaders(303, -1);
        ex.close();
    }
}
```

9. Simple Architecture



10. How the Application Works

1. Start the Java server and open the application in the browser.
2. The login page asks for a username and password.
3. Java checks the username and password and starts a session for the user.
4. The user opens the Dashboard, Patients, Doctors or Appointments page.
5. When the user submits a form, the browser sends the request to the Java backend.
6. Java checks every field and adds or deletes the record in the ArrayList.
7. Java generates an HTML response (styled by style.css) and sends it back to the browser.
8. The browser displays the output.

11. How to Run

1. Install JDK 17 or later.
2. Create a folder named HospitalManagement.
3. Put Hospital.java and style.css inside it.
4. Open Command Prompt/Terminal in that folder.
5. Compile: javac Hospital.java
6. Run: java Hospital
7. The terminal will show: Hospital app running at http://localhost:8080
8. Open http://localhost:8080 in your browser and log in with admin / admin123.

12. Output 1 — Login Page

```
------------------------------------------------
 CityCare Hospital
Username: [ ]
Password: [ ]
[ Log in ]
Demo: admin / admin123 or reception / reception123
------------------------------------------------
```

13. Output 2 — Dashboard

```
Input:
Username: admin
Password: admin123

Output:
Dashboard
3 Patients    3 Doctors    2 Appointments
Today's appointments
#   Patient       Doctor           Date         Time    Reason
7   Rahul Kumar   Dr. Anita Rao    2026-11-02   10:00   BP check-up
8   Aarav Singh   Dr. Meera Iyer   2026-11-02   11:30   Follow-up
```

14. Output 3 — Add Patient

```
Input:
Name: Sneha Patel
Age: 28
Gender: Female
Phone: 9123400004
Condition: Migraine

Output:
Patients
#   Name          Age   Gender   Phone        Condition
4   Rahul Kumar   45    Male     9123400001   Hypertension   [Delete]
5   Priya Nair    32    Female   9123400002   Knee pain      [Delete]
6   Aarav Singh   7     Male     9123400003   Fever          [Delete]
9   Sneha Patel   28    Female   9123400004   Migraine       [Delete]
```

15. Output 4 — Book Appointment

```
Input:
Patient: Sneha Patel
Doctor: Dr. Vikram Shah
Date: 2026-11-03
Time: 09:30
Reason: Consultation

Output:
Appointments
#    Patient       Doctor            Date         Time    Reason
7    Rahul Kumar   Dr. Anita Rao     2026-11-02   10:00   BP check-up
8    Aarav Singh   Dr. Meera Iyer    2026-11-02   11:30   Follow-up
10   Sneha Patel   Dr. Vikram Shah   2026-11-03   09:30   Consultation
```

16. Output 5 — Delete Patient

```
Input:
Click [Delete] on patient 5 (Priya Nair)

Output:
Patients
#   Name          Age   Gender   Phone        Condition
4   Rahul Kumar   45    Male     9123400001   Hypertension   [Delete]
6   Aarav Singh   7     Male     9123400003   Fever          [Delete]
9   Sneha Patel   28    Female   9123400004   Migraine       [Delete]
```

17. Output 6 — Error Messages

```
Wrong username or password   -> Invalid username or password
Empty field (Condition)      -> Condition is required
Page opened without login    -> Redirected to the login page
Unknown page (/rooms)        -> Page not found.
```

18. Output Screenshots

Top row: login page, dashboard. Middle row: patients, doctors. Bottom row: appointments, server running in the terminal.

